Independent recompilation and axiom audit of the public Lean development on four-dimensional Yang–Mills existence: what was run, what passed, what was not checked

Kévin Rémondière
Individual reproducibility exercise with AI assistance

Version 1.1, 12 November 2026
(checks run on 11 November 2026; state of the lean4lean reruns read on 12 November 2026 at 05:20 CET)

Abstract

[computed] The recorded Linux build of `yang-mills-group/OpenAI-YM-20261028` completed successfully under the pinned Lean toolchain. [computed] A separately written axiom query and the supplied verification scripts passed on that build; their exact outputs and scope are recorded below. [computed] The Lean 4.32.2 rebuild and checks passed, and the two Lean 4.32.0 exports have the same recorded digest. [computed] The exported proof closure (145 151 declaration records) was submitted to three checkers other than Lean's own kernel (nanoda, nano-lean, logos-kernel), and a fourth, lean4lean, was run on the compiled modules. The unmodified nanoda replay ended with a capacity panic (exit status 101) and gave no verdict; a separate instrumented run of the same export finished 187 924 of 187 925 items (nanoda's own count of items) without error and left one unfinished. nano-lean reported "187925 attempted, 0 failures, 0 fallbacks" (35 s). logos-kernel returned code 0 and reported 180 988 declarations accepted; the difference from 187 925 is not explained here. lean4lean returned code 0 for 9 465 of the 9 470 modules and refused none: 9 463 in the first pass, and two of the seven modules left over from that pass (five stopped by the operator, one killed with code 137, one stopped by a memory guard) in reruns, one at a time, on a second rented machine. At the time of writing (12 November 2026, 05:20 CET) the other five are not decided: one was stopped by its three-hour limit, one is running and three have not started (Section 8.1). The common fast controls were replayed by a party that did not run the campaign; the outputs are identical to the recorded ones apart from durations, process numbers, memory figures and one file path. [read] The checked proposition is an existence statement; none of these checks concerns a mass gap, and the committee's own assessment says that the construction "provides no evidence for or against a mass gap". [read] This is a documentary reproducibility report, not an assessment of the mathematics. The report claims no human verification of the proofs.

1 Scope and non-claims

[read] This report was prepared for an individual researcher with AI assistance. The numbers come from the recorded executions of 11 November 2026 and, for the lean4lean reruns only, from logs read on 12 November 2026 at 05:20 CET (logs and return codes in the supplement); the text was written by reading those records, and the evening additions were audited sentence by sentence against their sources by a model of another family (see the statement on the use of AI at the end). The programs themselves were run by the campaign's agents; the fast controls were run again by a non-author (the replay paragraph near the end). The supplement contains our scripts as run, our Lean control files, the logs and the hashes; it contains no code of the repository under study, of the checkers other than Lean's own kernel, or of the kernels used.

[read] The status notation is [computed] for historical machine results transcribed from logs; [read] for statements read in a cited source at the stated location; [hypothesis] for assumptions;

[proved] for an argument proved here; and [heuristic] for a conjecture. No mathematical theorem is presented as proved here.

[read] The records of the evening of 11 November supersede the afternoon snapshot's pending statuses: the two builds and the nanoda execution are finished, the lean4lean first pass ended on 11 November with seven modules undecided (by 04:20 UTC on 12 November, two reruns had passed and five modules remained undecided), two further kernels were added, and the port to Lean 4.35 was stopped by the operator (Section 9).

[read] The question is whether the distributed formal statements were accepted in the documented executions, with which axiom dependencies. This does not establish that the definitions faithfully express the intended physical problem, the mathematical correctness of an informal account, or a mass-gap theorem. [read] No priority or human proof-verification claim is made. No human has re-read the proofs for this report.

2 What is established and what is not

[computed] The proposition whose proof is checked, as printed by Lean in our `Verif.lean` run, is `YangMills.JointExistence`: for every simple gauge group `g` there exist a Wilson family `C` satisfying `WilsonRegularity C` and such that, for every strictly increasing sequence `p`, there are a strictly increasing `q` and a hierarchy `T` with `T.Properties (p ∘ q)`. It is an *existence* statement. (`YangMills.JointExistence` is the proposition, a type; `YangMills.jointExistence` is the proof term whose axiom dependencies are queried.) The checks reported here show that the proof of this proposition (and of the other headline results, listed in the `VerifyJoint` output below) is accepted by the Lean kernel and, for the export or for the compiled modules, by the checkers other than Lean's own kernel described in Section 8 (with the limits stated there), with the three standard axioms for the headline results.

[read] They do not show that the formal definitions are the intended ones, and they say nothing about a mass gap: the checked proposition, as printed above, is not a mass-gap statement. The committee's assessment writes, in its list of what the construction provides (item 4, line 139 of the supplied text): "The construction provides no evidence for or against a mass gap in the theory." It also writes (lines 560–561): "The theorem excludes the specified finite free-field descriptions but does not rule out every description in terms of composites of Gaussian fields." This report adds no appraisal of the validity of the formal statement beyond what the checkers establish.

[read] *What this report does not establish.* It states that the published Lean development compiled in the recorded executions and that the named checkers accepted the numbers of items, declarations or modules stated, in the circumstances stated. It does not state that the mathematics is correct in the sense in which a human expert would judge it, it does not state that the formal statement is the right statement of the problem it is meant to express, and it says nothing about a mass gap.

3 Materials and execution conditions

[computed] The recorded checkout is commit `3456e5e75cf8d4cd558955a5c7638fba2d68f464` of https://github.com/yang-mills-group/OpenAI-YM-20261028; the historical manifest check reports `OK 9508 fichiers`. (The supplier's `SOURCES.sha256` has 9 507 lines, all correct at the replay; the repository contains 9 508 files outside `.git`, the 9 508th being `SOURCES.sha256` itself; our own list has 9 508 entries.) [read] The following are the recorded expected SHA-256 values of the principal inputs, not hashes recomputed while preparing this report.

File	Recorded SHA-256 [read]
Challenge.lean	d31b227db6e6e1f2aa5eaa66d3b004609fe7470dc2ac3825e0dce882a5fa4fe9
ChallengeSpec.lean	e48b0be2b72d16b9b87c6c1923b3ae814270387bf9312b5cb0661dfa3e243749
PublicTheory.lean	94111ad258b4218a263fcc0a7136446d5a6eff0e090809b72036fb3a7aecfff8
PublicWilson.lean	c1542c816cd42d23448b66b4cb021c7f3bdad64b64f1c643dde27147ce0c3233
Solution.lean	1838214ba7159a2babe703000cfd9c43905fb909c1f27b73c46aae48c11ece24
Construction.lean	86286ed17925b3e729cd2f721347e27054ea935f2653ceaa35b3b49fc25c343c
ConcreteSU2.lean	646bc0cbc0b3bd8ea3dac779f8a9dc587c5b95e8d933fcefe566178374ec4114
VerifyJoint.lean	f84e8b30a0dca3de42615ccca2e01f84037370f77799c77f77d206a8a54d3ec6
verification/CompactCheck.lean	52b1882fb70ed7e90a42107f577c79f7da807210874f278fd8efdd14de8d65b1
verification/Usage.lean	1e43b8b0b65c317df019cb5aef227bacea14a8eedf7fa9c40e4e4c17f2141da4
lakefile.toml	059b094901a965a455f46956a99f2f860aecf99b1d3e86d3eb51b8ed858bfe26
lake-manifest.json	389c72d12e69c19b8a699df76405b2b1538f8f46ae77837fd2e972dda52724c3
lean-toolchain	2773c517aa90b66ea8a2c52bddddf84393157797f8341be0df45294fff7fd32e

[computed] All downloaded archive checks reported `OK`. The expected release digests are below; the download source was https://github.com/leanprover/lean4/releases.

Lean	Commit prefix	Linux archive SHA-256 [computed]
4.32.0	8c9756b2	fca846f3588724a38ad19ee40292c67cb7438d7555903372e3308eef795ba516
4.32.2	f3b06c70	5f2069e6f5db73780f374ccb49ce8ea649aa20a0cebf0116816744c999ce72aa
4.35.0	eb6cafb7	7289597a44543246c9227b8cae501767e095ce17f6d448b95fc4a8dc97268f6d

[computed] The manifest digest was unchanged after dependency retrieval. Every dependency check below reported `CONFORME modifs=0`.

Dependency	Recorded revision [computed]
Cli	88679d088c9720c27ebdf2ba4dafe17341747f94
LeanSearchClient	c5d5b8fe6e5158def25cd28eb94e4141ad97c843
Qq	38d591e778f100aec9762bb582f9c7f55f50e9dc
aesop	a7dbf0c63b694e47f425f3dcddbc0e178bb432d3
batteries	023ce7d62a0531e22a5331e20b587817a80d49ff
importGraph	7e9612bf0b9ee66db3cb5b9988a35afc706f5a12
mathlib	81a5d257c8e410db227a6665ed08f64fea08e997
plausible	e12c1910fe855cbfc38803cd4e55543906d5fa62
proofwidgets	6e311e2a844da9b2cc3971187df2fe0066947b93

[read] The rented machine is recorded as an AMD Ryzen Threadripper PRO 7995WX with 96 cores, 192 threads and 251 GB of registered ECC memory (the figure of the rental offer), running an `ubuntu:22.04` container on host kernel `7.0.0-42`. These are the campaign's recorded machine descriptions, not new hardware checks. [read] The rented builds and both nanoda runs used this same machine. This first rented machine was destroyed by hand at 23:26 (local time) on 11 November 2026, after its logs had been copied; the lean4lean reruns of Section 8.1 ran on a second rented machine, on which the objects of the first 4.32.0 build had been restored from an archive (no hash comparison of the restored objects is recorded in the logs we read). [computed] The first controller recorded `coeurs=192 memoire=241Go TA=120 TD=96 TG=60`; the memory log read 243 GB available at the start of the first build. The records do not say which of 251 GB and 241 GB is the memory visible inside the container.

[read] The rented executions used the unprivileged `lean` user in a disposable container. [computed] The isolation log explicitly says `AUCUNE (espaces de noms indisponibles dans ce conteneur)`. [read] Network isolation was unavailable: this was not a hermetic build. The preparation retrieved the official dependency cache, and the rented Rust build used `-locked` with network available. Comments in reused scripts describing an offline sandbox do not establish that property for this container.

4 Build and the morning incident

[read] The project build started with no project `.lake/build` directory. Dependencies could use the pinned Mathlib cache. The command was `lake build WilsonReference WilsonConstruction`.

Recorded item	Result [computed]
Completion	`Build completed successfully (18126 jobs).`; `RC=0`
Raw start / finish	`2026-11-11 11:43:30` / `2026-11-11 12:51:26`
Elapsed interval	1 h 07 min 56 s, calculated from the raw markers
Project `.olean` files	9,470
Empty files / failures / code 139	0 / 0 / 0

[read] The raw markers are those of the files `build_p1.debut` and `build_p1.fin`; the narrative log gives the corresponding interval as 12:43:30–13:51:26 (local time). The arena record states that the rented machine's clock is UTC and that local time is one hour later, which is consistent with this offset. The narrative log rounds the elapsed interval to "1 h 08".

[read] In the morning, the home PC (Intel i5-14600KF) ran the same build targets in a `systemd-run` sandbox with network disabled. The first pass returned code 1 after 1 h 53 min 26 s, with four Lean segmentation faults (exit code 139). During the retry, Linux reported `kernel BUG at fs/buffer.c:3044!` in `free_buffer_head`, in the `kswapd0` task, and the machine restarted. The kernel version string was `7.0.0-42`, the same as that of the rented host. The retry reached job 17717 of 18126 and has no return-code record; three of the four modules that had segfaulted compiled in the retry with an unchanged 8 MB stack. Post-restart checks found the source hashes intact and empty files among the build outputs (95 empty files among 1 113 written during the retry); those outputs were not used for the rented audit. The incident's cause is not established here and is not attributed to repository code.

5 Axiom audit and supplied verification scripts

[read] Our separately written `Verif.lean` imports `ConcreteSU2`, queries axiom dependencies, and checks that the submitted term inhabits both the copied public target and its copied unfolded definition (`ChallengeSpec.lean`, lines 186–188). This is independent of the supplied checking scripts; it still uses their imported environment and the same Lean kernel. [computed] Its recorded return code is 0, and its axiom output is exactly:

```
'YangMills.jointExistence' depends on axioms: [propext, Classical.choice, Quot.sound]
'YangMills.constructed' depends on axioms: [propext, Classical.choice, Quot.sound]
'YangMills.su2Theory' depends on axioms: [propext, Classical.choice, Quot.sound]
```

[read] The deliberately false Lean witnesses assert the same false equality, one using `sorry` and the other an added axiom. They test the sensitivity of the axiom query, not whether Lean rejects an explicitly assumed axiom. [computed] This command also returns 0; the warning and dependency lines expose both witnesses:

```
/work/travail_4320/journaux/Temoin.lean:5:8: warning: declaration uses ‘sorry‘
'temoinSorry' depends on axioms: [sorryAx]
'temoinAxiomeUtilise' depends on axioms: [temoinAxiome]
```

Supplied script	Recorded result	Return code
`CompactCheck.lean`	[computed] Compact-estimate checks passed	0
`VerifyJoint.lean`	[computed] Type/axiom/replay checks passed; 32 min 48.46 s; maximum RSS 29,174,984 kB	0
`Usage.lean`	[computed] Usage census completed; 1 min 48.34 s	0

[computed] The complete `verification/CompactCheck.lean` output is:

```
COMPACT_ESTIMATE YangMills.CompactCheck.compactCoefficient [propext, Classical.choice, Quot.sound]
COMPACT_ESTIMATE YangMills.CompactCheck.compactRemainder [propext, Classical.choice, Quot.sound]
```

[computed] The complete `VerifyJoint.lean` output is:

```
TYPE_MATCH YangMills.jointExistence
ENDPOINT_AXIOMS YangMills.jointExistence [propext, Classical.choice, Quot.sound]
ENDPOINT_AXIOMS YangMills.constructed [propext, Classical.choice, Quot.sound]
ENDPOINT_AXIOMS YangMills.su2Simple [propext, Classical.choice, Quot.sound]
ENDPOINT_AXIOMS YangMills.su2Theory [propext, Classical.choice, Quot.sound]
```

```
ENDPOINT_AXIOMS YM.Nonperturbative.existence [propext, Classical.choice, Quot.sound]
ENDPOINT_AXIOMS YM.Nonperturbative.continuum [propext, Classical.choice, Quot.sound]
ENDPOINT_AXIOMS YM.Nonperturbative.continuum_with_position_tests [propext, Classical.choice, Quot.sound]
ENDPOINT_AXIOMS YM.Contacts.Solstice.celestial [propext, Classical.choice, Quot.sound]
ENDPOINT_AXIOMS YM.Pert.Position.finite_leading [propext, Classical.choice, Quot.sound]
ENDPOINT_AXIOMS YM.Pert.Position.normalized_four [propext, Classical.choice, Quot.sound]
ENDPOINT_AXIOMS YM.Pert.Parity.parityEndpoint [propext, Classical.choice, Quot.sound]
ENDPOINT_AXIOMS YM.Pert.Parity.low_read [propext, Classical.choice, Quot.sound]
ENDPOINT_AXIOMS YM.Pert.Asymptotics.comparison [propext, Classical.choice, Quot.sound]
ENDPOINT_AXIOMS YM.SharpGram.habitat [propext, Classical.choice, Quot.sound]
ENDPOINT_AXIOMS YM.SharpGram.inletFirst [propext, Classical.choice, Quot.sound]
ENDPOINT_AXIOMS YM.SharpGram.outletFirst [propext, Classical.choice, Quot.sound]
ENDPOINT_AXIOMS YM.Independent.Core.point [propext, Classical.choice, Quot.sound]
REPLAY_START project=100168 external=14364
REPLAY_PASS project=100168 external=14364 endpoints=17 duplicateAliases=0
Entering kernel replay with 100168 project declarations and 0 checked duplicate aliases
```

[read] According to the earlier source analysis (§3.2), `VerifyJoint` compares the reference and solution types and reachable public definitions, audits axiom dependencies and the external frontier, and replays the project closure in a fresh environment initialized with the pinned external libraries. It does not independently recheck those external library objects. Its replay uses the same Lean kernel; it is distinct from the experiment with other checkers below. `CompactCheck` checks public corollaries; `Usage` counts dependencies. No new upstream Comparator result is claimed by this report.

6 Usage statistics

[computed] The complete `verification/Usage.lean` output is:

```
USAGE_PASS headlines=17 project=250319 used=100168 outside=150151 modules=9469 modulesWithNoUsedDeclaration=2946
```

Census item [computed]	Count
Headline results	17
Project declarations in the census	250,319
Declarations used by those results	100,168
Declarations outside that dependency closure	150,151
Modules in the usage census	9,469
Modules containing no used declaration	2,946

[read] These are counts relative to the selected headline dependency closure, not a logical test or a judgment about the value of declarations outside it. [hypothesis] The usage-census module population and the build-output population are kept separate; their counts are not assumed to measure identical sets.

7 Two observations on the released records

[read] *Two digests recorded for one file in the release's verification folder.* For the file `Public Theory.lean`, which defines the structure `Axioms` used by the challenge statement, four records of the release carry two different digests: `challenge.sha256` (line 1) and the top-level `SOURCE S.sha256` (line 10) give `94111ad2...fff8`, the SHA-256 digest of the distributed file, whereas `verified-sources.sha256` (line 1) and the supplier's log `joint-check.log` (line 5) give `3940e8 dd...a55`. The other 19 files of that list agree. The pilot reports finding no matching file among the repository objects searched in the two recorded commits (one commit each); the documented text searches found no explanation in the release or assessment. These searches do not establish absence of such bytes from all repository objects. [read] The pilot records that none of 37 simple transformations of the distributed file (line endings, final newline, trailing blanks, byte-order mark, Unicode normalisations, removal of each documentation comment in turn) reproduces the second digest; these trials are recorded in the campaign journal (entry of 11 November, 16:45:51), which is

not part of the supplement, and no script or output of them is included. What can be concluded is narrow: for this one file, the supplier's published log does not record the digest in the distributed-source manifest. What, if anything, differed in the checked source cannot be determined from these records. The committee's replay and ours used the distributed file.

[read] *A declaration-generating metaprogram.* The file `YM/UVExtension/Orchard/Barrel.lean` (71 lines) defines a command `vintage` which copies the type and the proof of an existing declaration, replaces constants by variants in a namespace `Vintage`, and submits the result to `Lean.addDecl`, hence to the kernel; according to the pilot's reading of its code, it refuses constants marked unsafe. For the headline results, the axiom query of Section 5 reports only the three standard axioms. The pilot's text search of `YM/` and the 13 root construction files identifies this as the only file importing or opening Lean's metaprogramming interface, and reports no `native_decide`, `implemented_by` or `debug.skipKernelTC` match, and no `opaque` or `unsafe` at the beginning of a line (the word `unsafe` occurs in an error message of `Barrel.lean`, line 63, and in one comment). [computed] The supplier's usage table lists 208 declarations in the namespace `Vintage`, in five modules, 174 of which are marked as used. Of these, 156 (146 used) belong to the four modules containing command-generated outputs; the remaining 52 (28 used) are explicit structures and their companion declarations in `Orchard.First`. The used rows record `YM.SharpGram.habitat` as their single headline witness; this column does not establish exclusive dependence on that endpoint. [computed] The separate search of the exported challenge closure (145 151 declaration records) reports no `Vintage` name component. [read] The committee's assessment mentions generated declarations in general terms; the documented search finds no `vintage`, `metaprogram` or `addDecl` occurrence.

8 Checkers other than Lean's own kernel

[read] The exporter source was https://github.com/leanprover/lean4export, commit `4e7915201d3f9f04470d9eae002fa695f7cdc589` (the recorded release tag is `v4.32.0`); the checker source was https://github.com/ammkrn/nanoda_lib, commit `0da9f98feadd3a0a2ef4b3dbcf52a298a6fef929`, built with Rust 1.97.0. [read] The recorded exporter format is NDJSON 3.1.0; nanoda is version 0.4.32. [read] The earlier report (§3.4) describes nanoda as a Rust implementation using `num-bigint`, without the Lean runtime or GMP. This is implementation independence, not a claim that the checker has itself been proved correct.

[computed] The rented-machine executable digests are

Executable	SHA-256 [computed]
`lean4export`	`e57369980b0b81228580ce08066fb9bd738e717e002673a143f4956d217266b0`
`nanoda_bin`	`ce46eafff2c49ecef7e6ff17fd1e792c0ce9a3b7817af560b10f0286c3e5d2c1`

[read] The export requests `jointExistence`, `constructed` and `su2Theory`, with their dependency closure including external libraries and explicitly listed primitives. The declaration total is the script's count of NDJSON declaration records.

Export item [computed]	Recorded result
Return code / `PANIC` matches	0 / 0
Declaration records	145,151
Lines	58,605,083
Bytes	3,224,695,689
Elapsed time	3 min 48.32 s

[computed] The export's recorded SHA-256 is `f1ff87d70c21f2f2d620d23bb0596a6947cabc3e32ff7ec5725c7a0e7089a06c`.

[read] The checker-chain witnesses use the Lean prelude without project imports. Besides the true theorem and the axiom-bearing witnesses, the poisoning test changes the exported literal in the true theorem's type from 4 to 5. [computed] The replacement count is 1.

Witness	Exact diagnostic [computed]	Code
`temoinVrai`	`Checked 584 declarations with no errors`	0
`temoinSorry`	`Error: export file declares unpermitted axiom "sorryAx"`	1
`temoinAxiomeUtilise`	`Error: export file declares unpermitted axiom "temoinAxiome"`	1
`temoinPoison`	`assertion failed: self.def_eq(u, v)` (checker panic)	101

[read] The full-replay configuration permits only `propext`, `Classical.choice` and `Quot.sound`, makes any other declared axiom a hard error, and requests endpoint printing. An axiom printed as present by nanoda need not be used by a particular theorem; Lean's dependency query answers that separate question. [computed] The full replay by the unmodified nanoda binary (version 0.4.32, 24 worker threads) did not terminate within three hours: 23 workers had exited without any message, the export had been read entirely, and one worker remained active; the memory log recorded `nanoda=100 Go`. To identify what it was checking, a diagnostic build was made from a copy of the same sources: the recorded diff adds a timer and two progress prints around the unchanged `check_declar` call, and the same export was replayed with 32 workers. [read] The journal does not name the author of this patch; the attribution is given in the statement on the use of AI. [computed] Of the 187 925 items of the export as nanoda counts them (inductive types, constructors and recursors counted separately; they correspond to the 145 151 exported declarations), 187 924 finished checking, and exactly one remained unfinished when the diagnostic was stopped (exit status 143): `YM.PhysicalNext.Repeat.Luna.uniformE` (`YM/FullVolume/Luna/Ocho.lean`, line 136), whose proof uses a helper proved by `rfl`. The pilot's summary reports no error in that diagnostic log; the full compressed log is not included (size and SHA-256 in the supplement) and was not reopened for this report. `chrono.out` records Lean-kernel (4.32.0) times of 12 ms for a copy of that declaration, and 88 601 ms, 1 046 ms and 989 ms for `orbit_eq`, `whole_apply` and `key_orb`, respectively; the pilot records diagnostic nanoda times of 370.5 s, 173.8 s and 172.5 s for those three declarations. [computed] Kernel times depend on machine load: a later non-author replay of `Chrono_run.lean` on a less loaded machine reproduced exactly the node counts and measured 4 ms for `uniformE`, 17 672 ms for `orbit_eq`, 629 ms for `whole_apply` and 375 ms for `key_orb` (and 1 525 ms for `tensor_jet`, the fifth declaration of the file; its recorded time was 7 547 ms). No logical rejection is reported in the supplied replay records. No complete verdict was obtained from nanoda; an unfinished timed run does not establish non-termination (see the final state below). The other checkers of this section are reported after it.

[read] The committee's verification records list "No second-kernel (e.g. Nanoda) check." among their limitations; its assessment states that all three runs described there used Comparator's built-in Lean kernel. The replay reported here is thus an additional check.

[computed] The unmodified replay ended by itself after 4 h 12 min 54 s with exit status 101 and a maximum resident set recorded as `145061124 kbytes`. Its log records "`thread 'thread_9' (557491) panicked at src/util.rs:59:9: index 2147483648 exceeds 31-bit capacity`", followed by a backtrace-enabling note. [read] According to the pilot's reading of `nanoda_lib/src/util.rs:40-59`, internal pointers have 32 bits, 31 of which are index bits; that source was not read for this report. The reported failing index is exactly 2^{31}; the panic does not report the graph's actual entry count. The pilot attributes the panic to the declaration identified by the diagnostic run; the unmodified binary's log names the thread, not the declaration. [computed] The recorded termination is a capacity panic, with no logical rejection diagnostic, no success message and no printed target axioms. The 187 924 completed items out of 187 925 are the instrumented diagnostic's count; the unmodified replay supplies no final checked-item count or complete second-kernel verdict.

8.1 A third checker: lean4lean

[read] lean4lean (https://github.com/digama0/lean4lean, Apache-2.0) is a re-implementation of Lean's kernel written in Lean. We used revision `408edad8930c1d9d974f93fcbe4a304a174b2b6f`, whose `lean-toolchain` is 4.32.2 (no revision targets 4.32.0), with the 4.32.0 toolchain and without

any change to its code; the executable has SHA-256 `f3adb28da674b22c2848671d3791b25356ec3d41d7204767ef7783d533eb3d38`. [computed] Calibration: a true witness was accepted (return code 0), and two false witnesses (`False := True.intro` and `1 = 2 := Eq.refl 1`), made with `debug.skipKernelTC` so that Lean's kernel does not see them, were refused (code 1, "declaration type mismatch"). The checker is invoked on a module name and replays the declarations of that module in the environment obtained from the imports, which are read from the compiled objects and not rechecked at that step.

[computed] The module `YM.FullVolume.Luna.Ocho`, which contains `YM.PhysicalNext.Repeat.Luna.uniformE`, was accepted (code 0, 25 s, 7.2 GB, 172 declarations); so were the modules of the three other declarations that were heavy for nanoda (`orbit_eq`: 45 s; `key_orb`: 21 s; `whole_apply`: 22 s).

[computed] A pass over all 9 470 modules of the project (9 457 `YM.*` modules and the 13 root modules), one module per call on the compiled objects of the first 4.32.0 build, returned code 0 for **9 463 modules**, with no failure; the pass ended at 22:02:01 UTC on 11 November 2026, and the sum of the recorded durations of these 9 463 modules is 153 446 s. **Seven modules were not carried to completion in that pass**: five `YM.RemainderComparison.*` modules (`Flues.Hearth`, `Solos.Hues`, `Typeface.Cards`, `Typeface.Press`, `Valse.Pellets`), stopped by the operator (code 143) after 1 526 to 1 772 s, when the pass was restarted with bounds of 15 GB per process and 25 minutes per module (header of the restart script); `YM.RG08.Clef.Bravo`, killed (code 137 after 2 081 s; the log records only the word "Killed", not the cause); and `YM.Work.F05`, terminated (code 143) after 21 s by the memory guard of the restarted pass, which stops any lean4lean process above 15 GB. For these seven modules the logs of the first pass are empty and no refusal was recorded. They are being restarted one at a time on a second rented machine, with a nominal limit of 55 GB of resident memory and a timeout of three hours each, in the order of the table below. [computed] The results below were read in the logs of that machine on 12 November 2026 at 05:20 CET; copies are in the supplement (`supplement/lean4lean/relances/`).

Module	State when read	Seconds	Peak resident (kB)
`YM.Work.F05`	code 0; "checked 116 declarations"	32	21 558 616
`YM.RG08.Clef.Bravo`	code 0; "checked 14 declarations"	6 999	92 764 324
`YM.RemainderComparison.Valse.Pellets`	code 124: stopped by the three-hour timeout; no verdict	10 806	101 667 032
`YM.RemainderComparison.Typeface.Press`	running since 03:01:25 UTC (1 h 18 min 48 s when read); resident 71 101 692 kB	—	—
`YM.RemainderComparison.Typeface.Cards`	not started	—	—
`YM.RemainderComparison.Solos.Hues`	not started	—	—
`YM.RemainderComparison.Flues.Hearth`	not started	—	—

Hence 9 465 of the 9 470 modules returned code 0 at the time of writing; none returned a refusal; **five are pending**: one stopped by its timeout without a verdict, one running, three not started. These statuses describe only the snapshot read at 04:20 UTC on 12 November 2026; the reruns were still in progress. The second rented machine is recorded with 101 GB of memory (the pilot's log); two of the reruns exceeded the nominal 55 GB of the script, with peak resident sizes of 92 764 324 kB and 101 667 032 kB, the second being of the order of the whole recorded memory (we did not investigate what these peaks include); the script's memory check selects the first process whose command line matches the module name, which on the recorded process list is the `time` wrapper, so the check probably did not watch the lean4lean process; we did not examine the memory state of the machine during these runs. A timeout without verdict is neither an acceptance nor a refusal. Twenty module names are prefixes of other module names and the tool takes a name as a prefix, so these runs also recheck their submodules (redundancy, not error).

[read] Limits: lean4lean is written in Lean and runs on Lean's compiler and runtime, so a fault in that base is not independent; it follows the official kernel's algorithm closely; the imports of each module are taken from the compiled objects as they are; the objects are those of our own reference compilation (for the reruns, restored on another machine from an archive); the revision was not designed for 4.32.0; and the calibration shows that the chain works, not that lean4lean is correct.

8.2 Two kernels of the Lean Kernel Arena

[read] Two recent kernels listed in the Lean Kernel Arena (https://github.com/leanprover/lean-kernel-arena, commit `6c2d82164f1a2b1ac94aee62290fe78a3c3853fb`) were built from the arena's commits and run on the same export file (SHA-256 recomputed on the machine: `f1ff87d7...a06c`, identical). Both are MIT-licensed Rust programs; the binaries were built without the arena's profile-guided optimisation phase, so they are not the arena's exact binaries.

	nano-lean v0.3.3	logos-kernel [computed]
Commit	`5c8a523d`, `sdiehl/nano-lean`	`1e6ad5b7`, `Brahmastra-Labs/logicaffeine`
Binary SHA-256	`4193c491...2d75` (`nl-fast`)	`fad7f656...9cc4`
Command	`nl-fast -j 8 --axioms propext,Classical.choice,Quot.sound`	`logos_kernel --check sem --stats`
Return code	0	0
Result	"187925 attempted, 0 failures, 0 fallbacks"; the axiom policy refused nothing	"accepted: 180988 declarations, 58605083 records"
Time, memory	35 s, 3 890 036 kB	6 min 28.6 s, 7 089 736 kB
Calibration	true witness accepted (584 declarations); poisoned witness refused; `sorryAx` and added-axiom witnesses refused by the axiom policy (accepted without that option)	true witness accepted (462 declarations); poisoned witness refused; `sorryAx` and added-axiom witnesses accepted (no axiom policy)

[computed] For nano-lean, a one-thread rerun with tracing prints 187 925 "start" and 187 925 "end" lines, among them `YM.PhysicalNext.Repeat.Luna.uniformE` (10.8 ms), the declaration that nanoda did not finish; a run restricted to that declaration (`-declaration`) returned code 0 with "1 attempted, 0 failures" (it does not recheck the dependencies). logos-kernel prints no per-declaration trace, so it covers `uniformE` only through its global acceptance message; the gap between its 180 988 and the 187 925 items of the export (its own counter prints 187 925) is not verified here: the arena record suggests, without checking line by line, that the 6 937 missing ones are inductive types, constructors and recursors admitted after re-derivation.

[read] Limits: both kernels read the same export file as nanoda, so there is no independence from `lean4export` or from the compilation that produced it; "accepted" says that these kernels find the declarations of the closure well typed, not that the export matches the public statement (that is the `TYPE_MATCH` check of the Lean side) nor that the formal statement is the intended one; same machine, same operator; the calibration is minimal (for logos-kernel only the poisoned witness tests the logic; no mutated copy of the full export was tried); the arena's own suite lists two "declined" corner cases for nano-lean (no fault for logos-kernel); the memory guard set for the first two nano-lean runs watched the `time` wrapper and not the kernel (a limitation of our script; the measured peak was 3.9 GB); two other candidates that met the arena's conditions (palladium, kefir) were not run because the instruction limited the number to two; and the only external evidence on their soundness considered here is the arena's tests.

9 Other Lean versions and the partial port

[computed] The full rebuild under Lean 4.32.2 (commit `f3b06c705e6c85f5314019d5d3baab0fec5b580c`), with Mathlib and the other dependencies rebuilt from their sources, ended with return code 0: "Build completed successfully (18133 jobs)", no error, 9 470 compiled objects of the project and 8 639 of the libraries, in about 4 h 52 min on a heavily loaded machine. Under this version, our axiom query again reports only `propext`, `Classical.choice` and `Quot.sound` for `YangMills.jointExistence`, `YangMills.constructed` and `YangMills.su2Theory`; the two sabotaged witnesses are reported; the supplier's `CompactCheck` returns 0, and its `VerifyJoint` returns 0 in 35 min 54.58 s with `TYPE_MATCH YangMills.jointExistence` and `REPLAY_PASS project=100168 external=14364 endpoints=17 duplicateAliases=0`.

[computed] A second complete build under Lean 4.32.0, in a separate project directory (`/work/travail_4320_p2`) with the same pinned dependency cache on the same rented machine, ended with return code 0: 9 470 compiled objects, none empty, no failure. File by file, 8 426 objects are bit-identical to those of the first build; 1 044 differ. The pilot reports that these are exactly the objects that embed the absolute path of their source file (the two sets of file names coincide); the retrieved manifests alone do not establish that paths are the only differing bytes. Bit-identity of all objects would require building at the same absolute path. To remove this reservation for the challenge theorem, the closure of `YangMills.jointExistence`, `YangMills.constructed` and `YangMills.su2Theory` was exported with `lean4export` from each build. Both exports have 145 151 declarations, 58 605 083 lines and 3 224 695 689 bytes, and the same SHA-256 digest, `f1ff87d70c21f2f2d620d23bb0596a6947cabc3e32ff7ec5725c7a0e7089a06c`; neither contains the build path. Both builds used one machine and one toolchain binary: this is a repeatability check of the build, not an independent implementation of the kernel.

[read] Two distinct compatibility experiments used Lean 4.35.0. The first retained the original libraries; the second changed Mathlib to revision `56a005855ea17e0b0958f73ec801ae140b0a1679`, with the project code initially unchanged.

Configuration	Recorded outcome [computed]
Original dependency pins	Code 1 in 24 s; 11 library modules failed in Batteries, Mathlib, Qq and ProofWidgets.
Matching Mathlib, unchanged project	Code 1 in 8 min 29 s; 169 project `.olean` files produced; 12 project modules failed.

[read] The first trial did not reach the project code. The second exposed API and elaboration incompatibilities; dependent modules were not thereby tested. Its cache extraction was partial, with `No file descriptors available`, and remaining library work was rebuilt from sources.

[read] The subsequent port attempted to preserve statement text, names, binders, imports and options. The patch reports describe static edits, not completed compilations by their writer. Besides proof bodies, they explicitly record API adaptations in definition bodies `clockD`, `dg`, `cap` and `lump` (iterations 1 to 4) and `view` (iteration 15). Their equivalence arguments are read in those reports; they are not independently proved here. Thus “proof-only” would overstate the literal edit scope.

Stage	Edited files [read]	Edit regions [read]	Copied files [read]	Project objects [computed]	Failed modules [computed]
Unchanged	—	—	—	169	12
Iteration 1	12	17	12	230	14
Iteration 2	13	17	14	268	9
Iteration 3	8	14	9	354	17
Iteration 4	15	18	17	504	27

[computed] The builds of iterations 1 to 9 returned code 1. The later builds are recorded in the follow-up file and in the build records as follows (accumulated project objects; failed modules; return code, where 124 is the exit status of `timeout`, i.e. the build was cut by its time limit and did not complete):

Iteration	Project objects	Failed modules	Code
5	555	12	1
6	590	9	1
7	639	9	1
8	701	8	1
9	737	4	1
10	820	10	124
11	915	12	124
12	975	10	124
13	1 030	13	124
14	1 074	15	124
15	1 505	66	124

[computed] **The port is partial and was not continued**: it was not continued after iteration 15; the pilot's note gives cost as the reason, and the logs record neither the reason nor a stop time. The last build, that of iteration 15 (20:49:35 to 21:39:36 local time), ended on its time limit (code 124) with 1 505 project objects compiled out of 9 470 and 66 failed modules in that build; no iteration 16 was started. From iteration 10 on the builds ended on the time limit, so the failure counts of iterations 10 to 15 are not comparable with the earlier ones. The record of touched statements (up to iteration 12) has 17 entries; the reports of iterations 13 and 14 state that no statement was touched; the patch report of iteration 15 records a change in the body of the definition `view`. [read] Object counts are accumulated outputs, not numbers of newly repaired modules. Copied-file counts include unedited files; failure lists include persistent failures as well as newly reached modules. [read] The patch reports of iterations 3 to 6 leave `LocalPolynomial.lean` blocked by an obsolete identifier in a statement whose editing was prohibited, and those of iterations 1 to 8 leave `Orthogonal.lean` blocked by an unreadable working copy; the reports of iterations 9 to 15 mention neither file. These are different limitations; neither establishes a mathematical obstruction to porting. [read] No completion-time estimate is asserted; a complete port would be a separate piece of work.

10 Limits and reproduction

[read] This campaign's successful build and Lean checks establish recorded acceptance of the formal statements under the documented environment and assumptions. They do not independently establish the intended mathematical or physical meaning, recheck every cached external proof, validate hardware or compiler correctness, or supply a complete verdict from nanoda, which ended on a capacity limit. Three of the checkers other than Lean's own kernel accepted the closure or (for lean4lean) 9 465 of the 9 470 modules, and nanoda finished 187 924 of 187 925 items in an instrumented run (Section 8), with the limits stated there (same export file for three of them, same machine, five modules still undecided for lean4lean). The text has been read against its sources by models, not by a human (see the statement on the use of AI at the end).

[read] The committee describes a clean rebuild with no pre-existing project build artifacts (§9.1), and reuse of submission objects for its subsequent combined Comparator run (§9.2). Its own statement of the verification limits, §9.2, lines 761–765 of the supplied text, reads:

> Comparator kernel used Lean 4.34.0. On macOS, the runs used the upstream development launcher `fake-landrun.sh`, which runs the checks without Linux Landrun's process isolation. They therefore record the statement, definition, axiom and kernel checks, but do not test isolation of potentially hostile build code. An additional external kernel implementation was not used. Each verification record identifies the exact target, inputs and configuration that passed.

[read] Those Comparator runs and the submitted `VerifyJoint` replay are distinct checks. The rented Linux run reported here also lacked network isolation, and its nanoda verdict remains incomplete after the capacity panic.

Use of the construction by others

[read] Version 5.0.0 of the paper "The Yang–Mills mass gap from one bound at one scale" by D. Bhattacharjee, U. Bhattacharya, J. M. Lockwood, T. Romanov, I. J. Sessford and S. Singha Roy (Zenodo record 10.5281/zenodo.24147967; the record's creation time is 2026-11-11 23:29:13 UTC) refers to the construction examined here. The following sentences are quoted from the plain-text extraction of that version that we hold (line numbers of that extraction; the reference [45] is the repository examined here and [62] the assessment of the Yang–Mills group of 10 November 2026):

- lines 135–137: "... this is hypothesis (W), stated in full in Definition 7.2; it is the shape of the conclusion claimed by the unrefereed Lean 4 construction [45, 62], on which nothing below depends."
- lines 143 and 175: Theorem E begins "Under (W), (U) and (D_ℓ) ..." and Theorem F begins "Under (W) and (B_*) at one physical time extent t_0 ...".
- lines 285–287: "For the theories built on tori, the statement of the Clay problem reduces, by Theorem E with the construction of [45] taken as the existence input, to (Z) or (U) together with (D_ℓ) ...".
- lines 1069–1072: "A construction of this kind, for every compact simple group, is claimed in the Lean 4 development [45], assessed in [62]; neither is a refereed publication, the theorem below does not depend on them, and they are cited only as the source of the shape of (W)."
- lines 9788–9789 (Table 3, status of (W)): "input from [45, 62], unrefereed; not proved here".

[read] This report quotes these sentences and nothing more: it does not check that paper's statements, nor how they use the construction, and the checks reported here concern the Lean development, not that paper.

Replay of the fast controls by a non-author

[computed] During the night of 11–12 November 2026, an agent that had neither written the report nor launched the campaign replayed, from the files of the supplement, the controls that are fast, on the first rented machine (unprivileged user, the reference compilations used read-only, `LEAN_PATH` built by hand in the order given by `lake env printenv LEAN_PATH`, so that `lean` was called directly instead of `lake env lean`). Every output listed below is identical to the recorded one up to durations, process numbers and the path of one file name, except where a difference is noted in the table.

Control	Replay result
Source hashes	commit `3456e5e7`; 9 507 of 9 507 lines of the supplier's `SOURCES.sha256` correct; our list of 9 508 correct; `PublicTheory.lean`: 94111ad2...fff8 recomputed, the two hash lists of `verification/` disagree as stated above (19 of 20 lines agree)
Versions and tools	the three toolchain commits, the nine dependency revisions, the two tool binaries and the 9 470 compiled objects (4.32.0 and 4.32.2) as recorded
`Verif.lean`, 4.32.0 and 4.32.2	the three theorems depend on the three standard axioms; identical outputs
False witnesses, both versions	`[sorryAx]` and `[temoinAxiome]` reported
`CompactCheck`, both versions	code 0, identical two lines
`VerifyJoint`, 4.32.0	code 0 in 19 min 36 s (recorded: 32 min 48 s), 28 414 248 kB; identical output
`Usage`, 4.32.0	code 0, identical `USAGE_PASS` line
Export	code 0; SHA-256 `f1ff87d7...a06c`, 3 224 695 689 bytes, 58 605 083 lines, 145 151 declarations, 0 PANIC, no `Vintage`, no build path
nanoda witnesses	584 declarations accepted; the three false witnesses refused with codes 1, 1 and 101
`Chrono_run.lean`	node counts identical; durations shorter by factors from 1.66 to 5.01 (Section 8)
`Vintage` count	208 declarations (174 used) in five modules; 156 (146 used) outside `Orchard.First`; one headline witness for the 174; one file importing `Lean` among `YM/` and the 13 root files

[read] Not replayed by the non-author: the complete builds, the comparison of the compiled objects, the full and the diagnostic nanoda runs, the lean4lean pass and reruns, the 4.35.0 trials and the port, the 37 trial transformations of `PublicTheory.lean`, and the reading of the nanoda source mentioned above. The replay used the same machine, the same reference objects and the same tool binaries as the campaign, and was done by one agent.

Documented commands

[read] The following commands document the historical component invocations in the uploaded `/work` layout, as user `lean`, after root setup and source/tool preparation. The Lean 4.32.0 builds use fresh project build directories and retain the pinned dependency cache; the 4.32.2 copy excludes dependency build objects. The recorded shell stack setting was `ulimit -s 1048576`. The fast controls among them were replayed by a non-author (see the replay paragraph near the end); the others were not rerun.

```
bash /work/prepare.sh
ulimit -s 1048576
cd /work/travail_4320/OpenAI-YM-20261028
env PATH=/work/tc/4.32.0/bin:/usr/bin:/bin HOME=/work/home \
    LEAN_NUM_THREADS=120 LC_ALL=C.UTF-8 \
    bash /work/outils/interne_build_passe.sh p1
env PATH=/work/tc/4.32.0/bin:/usr/bin:/bin HOME=/work/home \
    LEAN_NUM_THREADS=8 LC_ALL=C.UTF-8 \
    bash /work/outils/interne_verifs_copie.sh 4320
env PATH=/work/tc/4.32.0/bin:/usr/bin:/bin HOME=/work/home \
    LEAN_NUM_THREADS=4 LC_ALL=C.UTF-8 \
    lake env lean -j1 -DmaxHeartbeats=0 verification/Usage.lean
```

[read] The build wrapper executes `lake build WilsonReference WilsonConstruction`; the verification wrapper executes the following commands, preserving stdout, stderr, timing and return-code records:

```
lake env lean -DmaxHeartbeats=0 -DmaxRecDepth=4096 /work/travail_4320/journaux/Verif.lean
lake env lean /work/travail_4320/journaux/Temoin.lean
lake env lean -j1 verification/CompactCheck.lean
lake env lean -j1 -DmaxHeartbeats=0 -DmaxRecDepth=4096 VerifyJoint.lean
```

[read] The invocations of the chain with checkers other than Lean's kernel, with the endpoint arguments actually recorded, are:

```
cd /work/export
env PATH=/work/tc/4.32.0/bin:/usr/bin:/bin HOME=/work/home \
    LEAN_NUM_THREADS=4 LC_ALL=C.UTF-8 \
    bash /work/outils/interne_temoins_nanoda.sh
cd /work/travail_4320/OpenAI-YM-20261028
env PATH=/work/tc/4.32.0/bin:/usr/bin:/bin HOME=/work/home \
    LEAN_NUM_THREADS=8 LC_ALL=C.UTF-8 \
    bash /work/outils/interne_export.sh joint \
    YangMills.jointExistence YangMills.constructed YangMills.su2Theory
cd /work/export
env PATH=/work/tc/4.32.0/bin:/usr/bin:/bin HOME=/work/home \
    LEAN_NUM_THREADS=4 LC_ALL=C.UTF-8 \
    bash /work/outils/interne_nanoda.sh joint 24 \
    YangMills.jointExistence YangMills.JointExistence YangMills.su2Theory
```

[read] The last command passes `YangMills.JointExistence` (the proposition) as second endpoint name where the export used `YangMills.constructed`; this is what was recorded. The unmodified run ended on the capacity panic and printed no endpoints (its print file is empty); in the configuration file written by the script (`interne_nanoda.sh`) these names only fill the list `pp_declars` (declarations to print); we did not read the nanoda source for this point.

[read] The alternative-version commands, after the separate-copy preparation in `pilote2.sh`, are:

```
cd /work/travail_4322/OpenAI-YM-20261028
env PATH=/work/tc/4.32.2/bin:/usr/bin:/bin HOME=/work/home \
    LEAN_NUM_THREADS=64 LC_ALL=C.UTF-8 bash /work/outils/interne_build_4322.sh
cd /work/travail_4320_p2/OpenAI-YM-20261028
env PATH=/work/tc/4.32.0/bin:/usr/bin:/bin HOME=/work/home \
    LEAN_NUM_THREADS=80 LC_ALL=C.UTF-8 bash /work/outils/interne_build_passe.sh p2
cd /work/travail_4350/OpenAI-YM-20261028
env PATH=/work/tc/4.35.0/bin:/usr/bin:/bin HOME=/work/home \
    LEAN_NUM_THREADS=8 LC_ALL=C.UTF-8 timeout 3600 lake build WilsonReference
cd /work/travail_4350m/OpenAI-YM-20261028
env PATH=/work/tc/4.35.0/bin:/usr/bin:/bin HOME=/work/home \
    LEAN_NUM_THREADS=40 LC_ALL=C.UTF-8 \
    timeout 5400 lake build WilsonReference WilsonConstruction
```

[read] The comparison is a diff of the two sorted, relative-path SHA-256 manifests:

```
diff /work/travail_4320/journaux/empreintes_olean_p1.txt \
     /work/travail_4320_p2/journaux/empreintes_olean_p2.txt
```

[read] The full source inventory, checksums and actual controllers remain necessary reproduction materials. The invocations above cover the historical controllers; the later diagnostic instrumentation and second export are documented in the supplement (`nanoda_diagnostic/`, `lean4lean/`, `arene/`). The lean4lean pass calls `lean4lean <Module>` once per module with `LEAN_PATH` set to the reference compilation (script `lean4lean/complet/worker.sh`); the arena kernels are called as in the table above (scripts in `arene/`). The supplement's `README.md` says how to replay each control.

Statement on the use of AI

[read] The report was first written by GPT-6.1 Sol (OpenAI) from the campaign's recorded logs, and completed with the evening results by Claude Fable 5.2 (Anthropic), the pilot of the campaign. The pilot's evening additions were then audited sentence by sentence against their sources by GPT-6.1 Sol (33 statements checked: 19 confirmed, 7 wrong and 7 not verifiable from the files; the report was corrected afterwards). The fast controls were replayed, and the supplement was assembled, by Claude Sonnet 5.5 (Anthropic). The two arena kernels were installed and run by an agent of the campaign; the arena record names only "the arena agent", and the attribution of that work to Claude Sonnet 5.5 is that of the project's session records. The campaign's agents included Opus 5.5 (an earlier reproducibility draft). The patch that prints progress in nanoda was written, according to the project's session records, by the pilot and reviewed by no model of another family; the journal

does not name its author. The port patches were written by GPT-6.1 Sol, with no separate model review. Version 1.1 corrects version 1.0 after a fact-check of the text against the records by a local model (Qwen 3.8), a reading by DeepSeek V4.1 Flash, the correction by Claude Sonnet 5.5, and one reading of the corrected text by GPT-6.1 Sol against the journal. These models did not recheck the mathematical proofs; the machine checks are those described above. This report does not identify the model writers or reviewers of the upstream mathematical proofs. The human author read the final text of this note before its release; he did not verify the proofs, and no human expert has checked them.